# Dependencies

*Version: 1.0.0 | Updated: 2026-04-30*

> *AIStudio — what's installed, why, and what's deliberately not (yet). This document is the narrative companion to* `requirements.txt`.

---

## Design Philosophy

Dependencies are chosen conservatively. Every package in `requirements.txt` earns its place. Commented-out items are documented here as deliberate deferrals — not oversights — with notes on when and why they would be added.

This matters for a project whose purpose includes understanding the stack at the implementation level. Pulling in a framework that abstracts away the hard parts defeats the point.

---

## Core Web & API

**`fastapi`**

The web framework powering all backend endpoints ( `/ask` , `/corpus/*` , `/health` , `/debug/*` , `/prewarm` ). Chosen for native async support, automatic OpenAPI documentation generation, and tight Pydantic integration that makes request/response validation nearly free.

**`uvicorn[standard]`**

The ASGI server that runs FastAPI. The `[standard]` extra pulls in `websockets` and `httptools` for faster HTTP parsing. Run with `--reload` during development for automatic restart on file changes.

`ollama`

Python client for the Ollama service running locally on port 11434. Used to send prompts to the LLM and receive streamed or batch responses. Ollama itself runs as a separate background service — this client is the bridge between FastAPI and that service.

`python-multipart`

Enables FastAPI to parse `multipart/form-data` — the encoding used when uploading files. Required for the corpus file upload endpoint.

---

## Vector Store

`qdrant-client>=1.7.0`

Client for the Qdrant vector database running locally on port 6333. Qdrant replaced ChromaDB after ChromaDB crashed at 32,285 chunks. Qdrant is stable at 105,964 chunks with native metadata filtering, Rust-based memory model (near-zero GC overhead), and a production upgrade path (sharding, replication, quantization, gRPC).

See `docs/architecture_decisions.md` Decision #2 for full rationale.

### ~~ `chromadb` ~~ *(replaced by Qdrant)*

Original vector store. Replaced due to instability at scale — crashed at 32,285 chunks during SEC 10-K corpus ingest. Kept in `vectorstore/chroma_store.py` as a fallback path; selectable via `AISTUDIO_VECTORSTORE=chroma` env var. Not the active production path.

---

## Embeddings & Reranker

`sentence-transformers`

Provides the CrossEncoder reranker ( `cross-encoder/ms-marco-MiniLM-L-6-v2` , ~90MB). Installed March 2026. Wires into `rag_core.py` after vector retrieval, before prompt assembly — fixes vocabulary mismatch (e.g. "AI governance" ≠ "Firmwide Artificial Intelligence Risk and Controls Committee").

Also enables alternative embedding model benchmarking (e.g. `bge-large-en` vs `nomic-embed-text` ).

Previously deferred. Now active.

---

## Utilities

**`httpx`**

Async HTTP client for making requests to the Ollama REST API. The `requests` library is synchronous — in an async FastAPI application, `httpx` is the correct tool.

**`python-dotenv`**

Loads environment variables from `.env` at startup. Used for configuration values (model name, Ollama port, data paths) that should not be hardcoded.

**`pydantic`**

Data validation and serialization. Every request body, response schema, and config object is a Pydantic `BaseModel`. FastAPI uses Pydantic natively — one definition produces validation rules, OpenAPI schema, and serialization.

---

## Progress / UX

**`tqdm`**

Progress bars during document ingestion. Without it, indexing a large corpus produces no output. With it, you see chunk-by-chunk progress and ETA.

---

## Document Parsing

**`pypdf`**

Fallback PDF text extractor. Used when `pdfplumber` is unavailable. Does not handle scanned PDFs (image-only) — those require OCR. Retained as a fallback in `loaders.py` — if pdfplumber import fails, pypdf runs instead with no page markers.

**`pdfplumber`**

Primary PDF extractor. Added Beta (March 2026). Extracts text with page boundary awareness — inserts `[PAGE_N]` markers at each page boundary during extraction. These markers flow through the chunking pipeline into Qdrant payload (`page` field) and `chunk_id` format (`filename::page-N::chunk-M`), enabling page numbers in citations and the PDF viewer Open ↗ feature. Falls back to pypdf gracefully if pdfplumber is unavailable.

**`openpyxl`**

Reads Excel files (`.xlsx`, `.xlsm`). Extracts cell values sheet by sheet.

**`python-docx`**

Reads Word documents (`.docx`). Extracts paragraph text, table contents, and heading structure.

**`python-pptx`**

Reads PowerPoint files (`.pptx`). Extracts slide text, speaker notes, and text boxes.

**`beautifulsoup4`**

Parses HTML documents (SEC 10-K filings in HTML format). Strips tags to extract clean text. Known issue: XBRL structured data tags (`<ix:*>`) in SEC filings pollute chunks — stripping these is a Beta roadmap item.

---

## Development

**`pre-commit`**

Runs code quality hooks before every `git commit`. Configured in `.pre-commit-config.yaml` to run `ruff` (lint) and `ruff-format`. Catches issues before they reach CI. Run `pre-commit install` once after cloning.

**`pytest`**

Standard Python testing framework. Run via `make test`, `make test-unit`, or `make test-integration`.

**`pytest-cov`**

Coverage reporting. Run via `make coverage`. Current coverage: ~26%. Add `--cov-fail-under=N` in CI to enforce minimum threshold.

**`ruff`**

Fast Python linter and formatter written in Rust. Replaces `flake8`, `black`, and `isort`. Configured in `pyproject.toml`. Run via `make lint` or `make format`.

**`setuptools`** / **`wheel`**

Python packaging utilities. Required for building AIStudio as an installable package. Will be essential for the v1.0 one-click installer.

---

## Deliberately Excluded

**~~ `langchain` ~~**

Popular LLM orchestration framework. Not used by design.

AIStudio implements the retrieval pipeline, context assembly, citation extraction, and conversation memory directly. This is slower to build but produces genuine understanding of where RAG breaks down, how context windows become a constraint, and what observability you lose when orchestration is managed for you.

LangChain is the right choice for shipping a product quickly. It is the wrong choice for a project whose purpose is substrate knowledge.

**When to add:** if AIStudio becomes a platform others build on top of, LiteLLM (not LangChain) would be the right abstraction layer.

---

## Planned Additions

| Package | Purpose | Release |
|---|---|---|
| `litellm` | Unified abstraction for local + cloud LLMs — swap providers via config | v2.0 |
| ~~ `pdfplumber` ~~ | ~~Page-aware PDF chunking~~ | |

| Package | Purpose | Release |
| --- | --- | --- |
| | | ✅ Active — installed Beta |
| `pytest-asyncio` | Async test support for FastAPI endpoints | Beta |
| `structlog` | Structured JSON logging for query traces and latency | Beta |
| `rich` | Visual benchmark output — colored tables, pass/fail markers | v2.0 |
| `pyyaml` | YAML benchmark question files — human-readable alternative to JSONL | ✅ Active — installed Beta |
| `uv` | Replace pip+venv — faster installs, better lockfiles, modern toolchain | v2.0 |